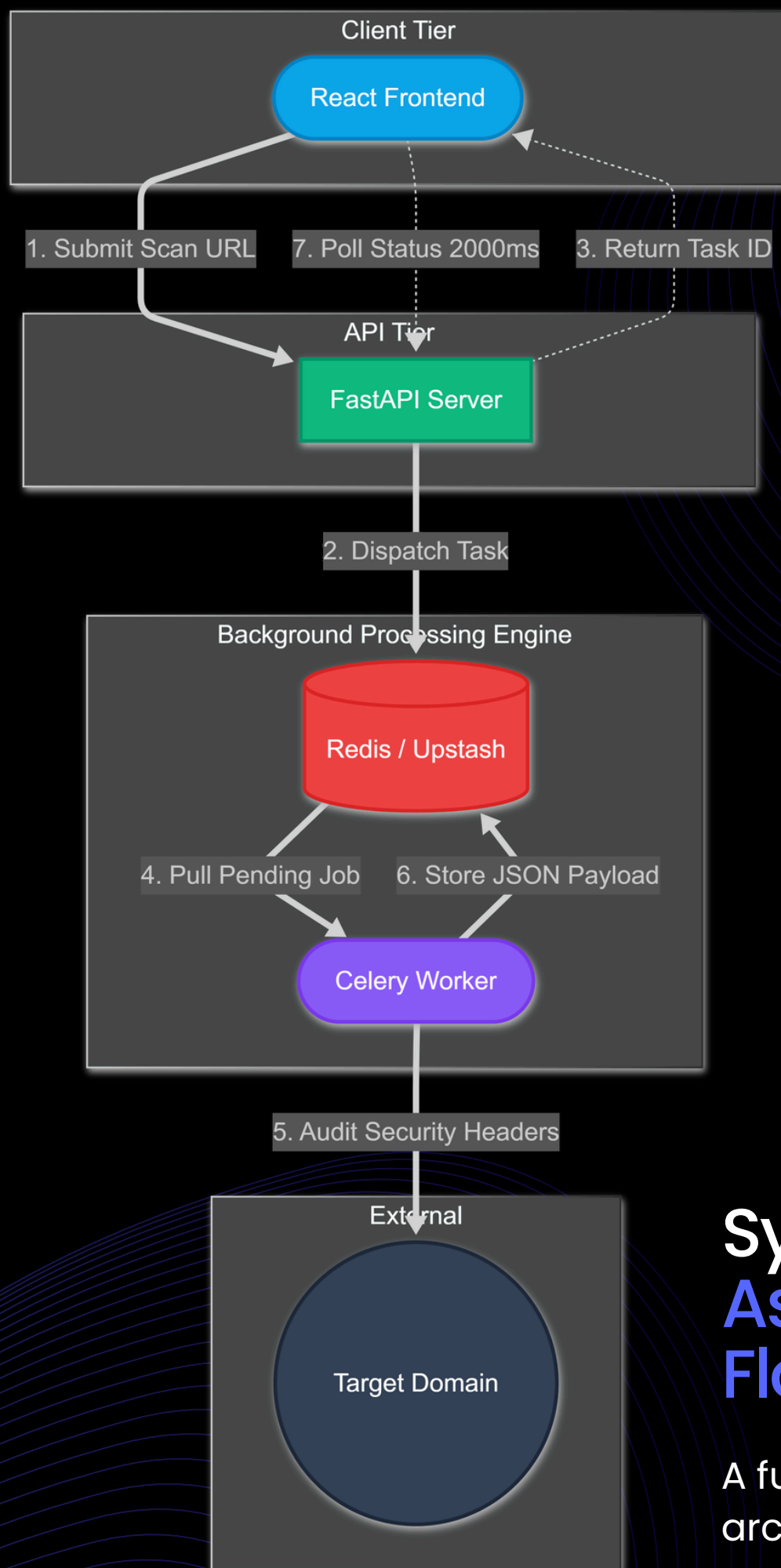


Async Edition

Cybersecurity Automation Platform

@Parakrama



System Architecture & Asynchronous Data Flow

A fully decoupled microservice architecture.

Help < > Cybersecurity Automation Project

headers_analyzer.py M main.py M X celery_app.py M queue_tasks.py M

cyber-platform-api > main.py > get_scan_status

```
1 from fastapi import FastAPI
2 from fastapi.middleware.cors import CORSMiddleware
3 from pydantic import BaseModel
4 from core.queue_tasks import run_headers_scan_task
5 from core.celery_app import celery_instance
6
7 app = FastAPI(title="Cybersecurity API - Async Edition")
8
9 app.add_middleware(
10     CORSMiddleware,
11     allow_origins=["*"],
12     allow_credentials=True,
13     allow_methods=["*"],
14     allow_headers=["*"],
15 )
16
17 class ScanRequest(BaseModel):
18     url: str
19
20 @app.post("/api/v1/scan/headers")
21 def dispatch_headers_scan(request: ScanRequest):
22     task = run_headers_scan_task.delay(request.url)
23
24     return {
25         "status": "success"
26     }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\acer\Documents\Cybersecurity Automation Project> cd cyber-platform-api
PS C:\Users\acer\Documents\Cybersecurity Automation Project\cyber-platform-api> venv\Scripts\activate
(venv) PS C:\Users\acer\Documents\Cybersecurity Automation Project\cyber-platform-api> uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\acer\\Documents\\Cybersecurity Automation Project\\cyber-platform-api']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [4564] using StatReload
INFO: Started server process [6464]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

ParakramaWelipitiya (1 day ago) Ln 31, Col 35 Spaces: 4 UTF-8 CRLF { } Python

The API Gateway (FastAPI)

Validates the incoming payload using Pydantic and instantly dispatches the job to the queue using Celery's `.delay()` method.

HelpCybersecurity Automation Project

headers_analyzer.py Mmain.py M Xcelery_app.py Mqueue_tasks.py M

cyber-platform-api > main.py > get_scan_status

1 from fastapi import FastAPI2 from fastapi.middleware.cors import CORSMiddleware3 from pydantic import BaseModel4 from core.queue_tasks import run_headers_scan_task

PROBLEMSOUTPUTDEBUG CONSOLETERMINALPORTS

- PS C:\Users\acer\Documents\Cybersecurity Automation Project> cd cyber-platform-api
- PS C:\Users\acer\Documents\Cybersecurity Automation Project\cyber-platform-api> venv\Scripts\activate

(venv) PS C:\Users\acer\Documents\Cybersecurity Automation Project\cyber-platform-api> celery -A core.queue_tasks w

info --pool=solo

[2026-05-27 15:34:18,040: WARNING/MainProcess]

Setting ssl_cert_reqs=CERT_NONE when connecting to redis means that celery will not validate the identity of the re

onnecting. This leaves you vulnerable to man in the middle attacks.

----- celery@Parakrama-Welipitiya v5.6.3 (recovery)

----- ***** -----

-- ***** ----- Windows-11-10.0.26200-SP0 2026-05-27 15:34:17

- *** --- * ---

- ** ----- [config]

- ** ----- .> app: cyber_scanner:0x283258578c0

- ** ----- .> transport: rediss://default:**@adapted-weasel-136628.upstash.io:6379//

- ** ----- .> results: rediss://default:**@adapted-weasel-136628.upstash.io:6379/

- *** --- * --- .> concurrency: 8 (solo)

-- ***** ----- .> task events: OFF (enable -E to monitor tasks in this worker)

----- ***** -----

----- [queues]

.> celery exchange=celery(direct) key=celery

[tasks]

. core.queue_tasks.run_headers_scan_task

[2026-05-27 15:34:19,434: INFO/MainProcess] Connected to rediss://default:**@adapted-weasel-136628.upstash.io:6379/

[2026-05-27 15:34:20,215: INFO/MainProcess] mingle: searching for neighbors

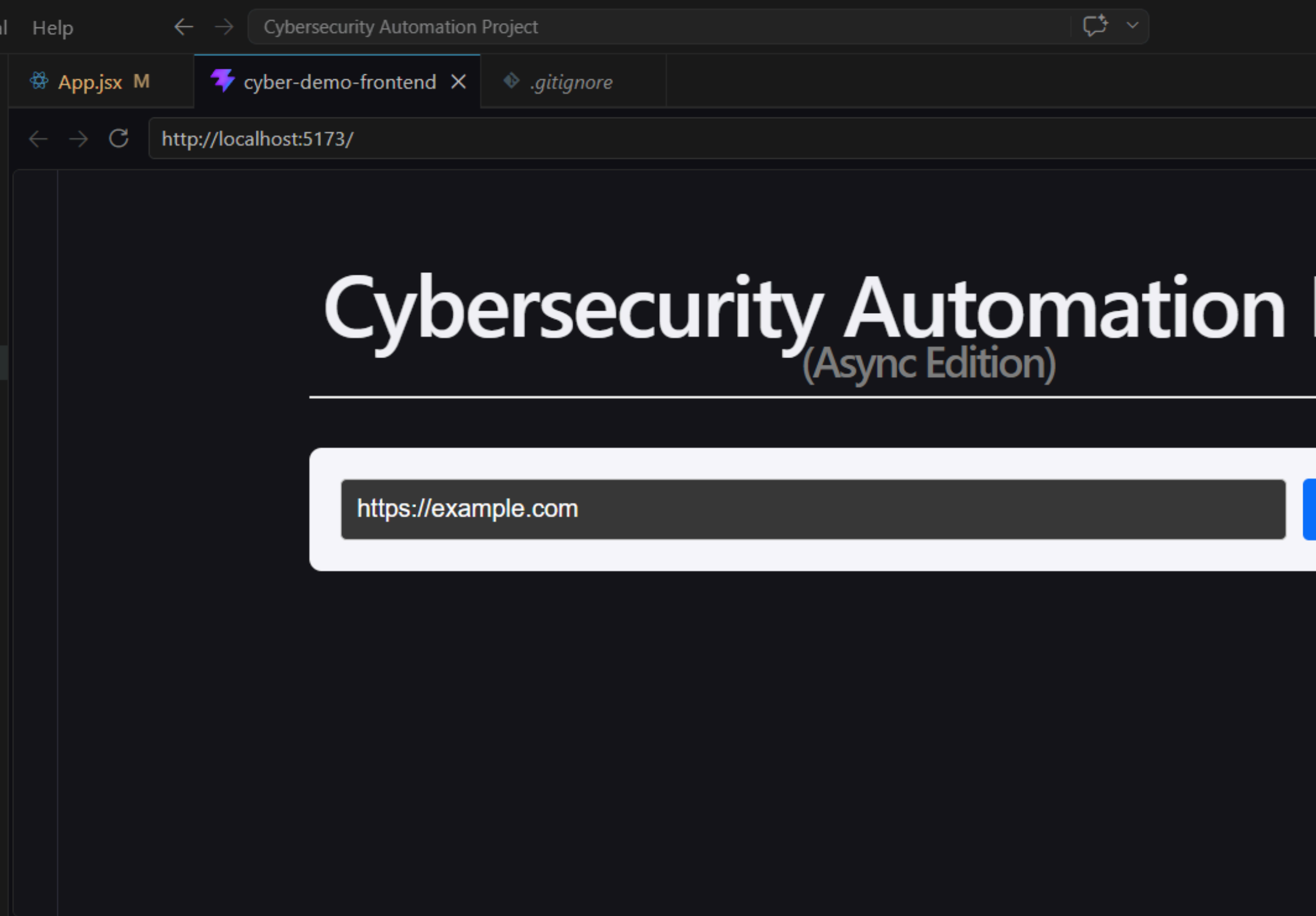
[2026-05-27 15:34:26,565: INFO/MainProcess] mingle: all alone

[2026-05-27 15:34:31,234: INFO/MainProcess] celery@Parakrama-Welipitiya ready.

ParakramaWelipitiya (1 day ago) Ln 31, Col 35 Spaces: 4 UTF-8 CRLF { } Python

The Worker Queue (Redis + Celery)

Message Broker: Cloud-hosted
Redis (Upstash) holds the
pending scan jobs.



PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

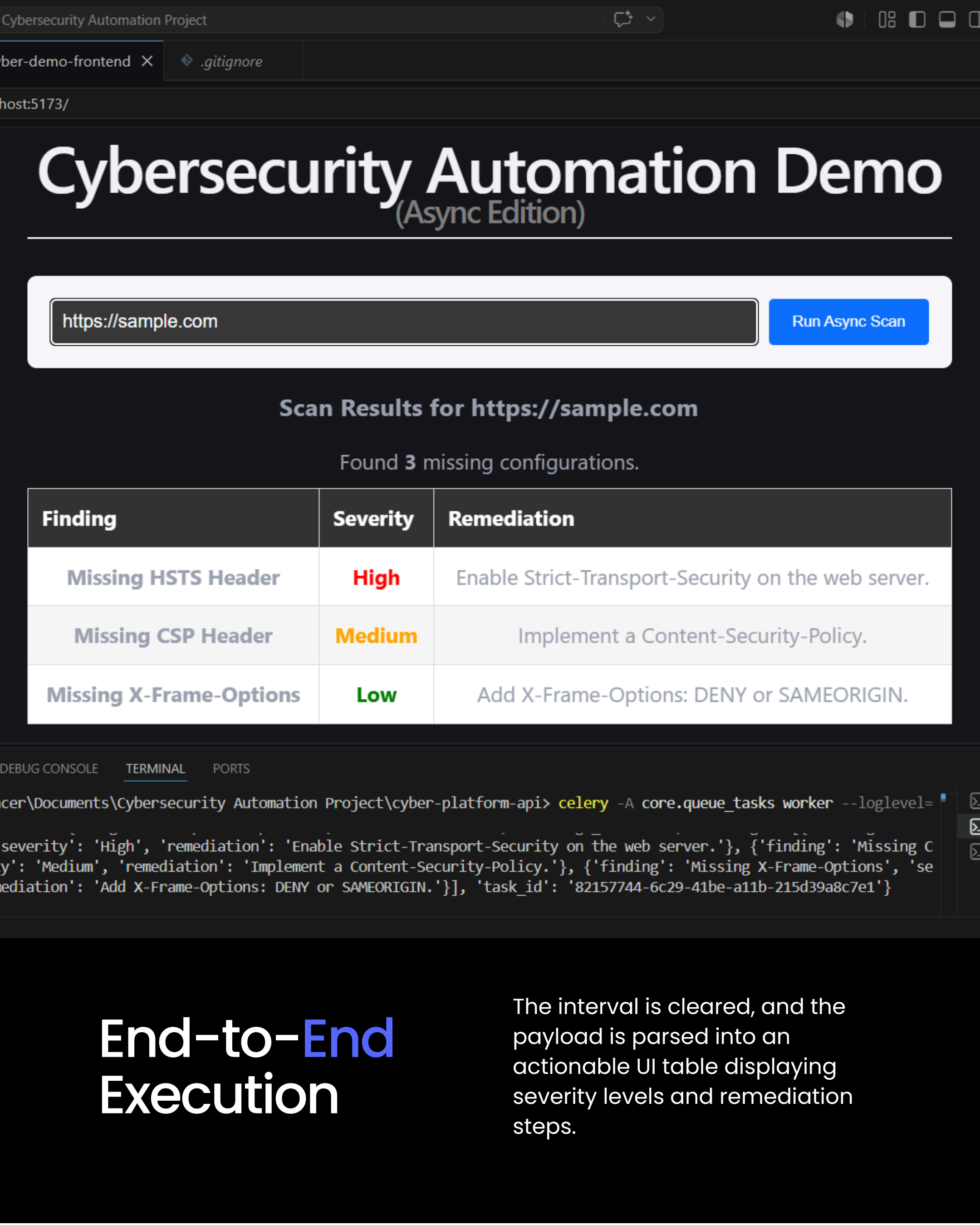
```
PS C:\Users\acer\Documents\Cybersecurity Automation Project\cyber-demo-frontend> npm run dev
VITE v8.0.14 ready in 362 ms
```

```
→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

```
█
```

The Frontend (React / Vite)

Submits the target URL to the FastAPI gateway.



Cybersecurity Automation Demo

(Async Edition)

https://sample.com

Run Async Scan

Scan Results for https://sample.com

Found 3 missing configurations.

Finding	Severity	Remediation
Missing HSTS Header	High	Enable Strict-Transport-Security on the web server.
Missing CSP Header	Medium	Implement a Content-Security-Policy.
Missing X-Frame-Options	Low	Add X-Frame-Options: DENY or SAMEORIGIN.

DEBUG CONSOLE

TERMINAL

PORTS

```
cer\Documents\Cybersecurity Automation Project\cyber-platform-api> celery -A core.queue_tasks worker --loglevel=
severity': 'High', 'remediation': 'Enable Strict-Transport-Security on the web server.'}, {'finding': 'Missing C
y': 'Medium', 'remediation': 'Implement a Content-Security-Policy.'}, {'finding': 'Missing X-Frame-Options', 'se
mediation': 'Add X-Frame-Options: DENY or SAMEORIGIN.'}], 'task_id': '82157744-6c29-41be-a11b-215d39a8c7e1'}
```

End-to-End
Execution

The interval is cleared, and the payload is parsed into an actionable UI table displaying severity levels and remediation steps.

Future Scalability SaaS Roadmap



Authentication & Persistence

Implement JWT-based login, domain verification via DNS TXT records, and PostgreSQL integration to permanently store users and historical scan data.



Module Expansion

Build and integrate the SSL/TLS Certificate Analyzer and aggregate findings into a unified 0-100 security score.



Security & Compliance

Wrap the Celery worker execution environment inside isolated,ephemeral Docker containers to safely execute active target scanning , and introduce API rate limiting at the FastAPI gateway.

So... Why automate security?



Because manual audits don't scale. By decoupling the scanning engine from the frontend, this platform proves that heavy security checks never have to compromise user experience.

[source code](#)